coding conventions resemble Open GL, and the library has the best multi-channel 3D positional sound support. The library is, currently, supported by Creative Technology and Apple, with an active community backing. It is an excellent DirectSound alternative.

The library models a collection of audio sources moving in a 3D space that are heard by a single listener somewhere in that space. The basic OpenAL objects are: a listener, a source, and a buffer. There can be a large number of buffers, which contain audio data. Each buffer can be attached to one or more sources, which represent points in the 3D space that emit some sound (audio). There is always one listener object (per audio context), which represents the position where the sources are heard—rendering is done from the perspective of the listener.

The source objects point to an audio buffer, besides the velocity, position, intensity and direction of sound. The audio buffer is the audio data. The listener object contains the velocity, position and direction of the listener (mostly your in-game character). With the help of these three objects, OpenAL helps recreate natural 3D sound in the virtual world (with natural effects such as the Doppler Effect). The library is built with an extensible architecture that makes it very agile and flexible for the future as well.

The API is highly cross-platform and binaries are available for the following platforms: Mac OSX, iPhone, GNU/Linux (OSS/ALSA), BSD, Solaris, IRIX, Windows, Xbox/360, AmigaOS 3.x, and MorphOS. New platform support is not likely in the near future, but this in itself is an exhaustive list.

And if you are wondering what games use Open AL, the following are just some examples: ***Battlefield 2***, ***Bioshock***, ***Colin McRae: DiRT***, ***Doom 3***, ***Ghost Recon: Advanced Warfighter***, ***Lineage 2***, ***Quake 4***, ***Race Driver: GRID***, and ***Unreal Torunament 2004***. The Open AL is also an integral part of the id Tech 3/4/5 and Unreal Engine 2/3 engines.

The library is licensed under LGPL. To work with Open AL, look at the detailed walkthroughs on Open AL by Dev Master (*www.devmaster.net/articles.php?catID=6*).

Blender

Once an in-house production software and later freed by its creators, Blender is the animation/CG industry's new kid on the block. Blender is a multi-platform 3D graphics application used for modelling, texturing, rigging, animating, etc. Its source code was opened and it is now released under the GNU GPL. It is also a very powerful tool when it comes to UV unwrapping, 2D/3D painting, skinning, sculpting, water-creation, physics, particle simulations... and no, the list doesn't quite end yet, but goes on to include post production tools as well, like a non-linear editing suite with a compositing tool to boot. Blender can also be used to create interactive 3D applications and games. It is available on multiple platforms including Linux, Windows, and Mac OSX. There are official and unofficial ports to other popular operating systems as well.

Blender has a light footprint and an intuitive interface, and covers almost all features of its commercial counterparts, even doing a better job at some places. Blender goes a step further to be a single resource for all your 3D production and post-production needs. It treads into real-time rendering and games with its integrated game engine. Not stopping there, Blender provides game logic, real-time physics and a workflow that connects your asset creation pipeline to your post-production and extends it to the realm of game development.

If you have worked on games or animation projects, you will have seen that both share a very similar development/production pipeline, only dividing towards the end, where one is rendered offline and the other in real-time. So it is an obvious choice to extend a well-built 3D production and animation suite to game development. Most of Blender is written in C/C++ and Python. It can be extended using plug-ins and can be customised using Python scripts as well.

Blender sports many of the features that are on par with 3D CGI production powerhouses, like Maya and Softimage. It supports a variety of geometric primitives, including polygon meshes, Bezier curves, NURBS surfaces, metaballs, fast subdivision surface modelling, digital sculpting, and outline fonts. It supports key-framed animation tools including inverse kinematics, armature (skeletal), hook, curve and lattice-based deformations, shape keys (morphing), non-linear animation, constraints, vertex weighting, soft body dynamics including mesh collision detection, LBM fluid dynamics, bullet rigid body dynamics, particle-based hair, and a particle system with collision detection. Blender renders geometry in all its glory – lit, textured and composited with the integrated YafRay ray tracer.

It has an intuitive interface, but it still can be a bit of a steep learning curve for even those who have experience in other 3D graphics applications like Maya or 3D Max, apart from those who have no previous experience. It will take almost the same amount of time to master Blender as it will to get comfortable with Maya and Max.

The Blender foundation tries to stay focused on creating cutting-edge tools and adding new functionality in Blender, while trying to make the learning process easy by conducting workshops, releasing DVDs, updating the wiki, tutorials and other documentation. The ever-growing community of Blender artists fuels the fire that burns in the hearts and minds of everyone involved with Blender.

Bullet Physics

Bullet Physics is a professional collision detection, rigid body and soft body dynamics library. The library is free for commercial use under the ZLib License. The library is under active development and its roadmap